

FundScan_Project_Document

FundScan – Comprehensive Project Document

Fund Performance & Investor Sentiment Analyst

Version: 1.0.0 | **Owner:** Stradit AI | **Audience:** Development Team

Table of Contents

1. [Project Overview](#)
 2. [Business Problem & Objectives](#)
 3. [System Architecture](#)
 4. [Technology Stack](#)
 5. [Infrastructure & Docker Services](#)
 6. [Database Design](#)
 7. [The 6-Phase Autonomous Pipeline](#)
 8. [Backend – API Reference](#)
 9. [Frontend – Dashboard & UI](#)
 10. [Observability Stack \(MLflow + Prometheus + Grafana\)](#)
 11. [Environment Configuration](#)
 12. [Developer Setup Guide](#)
 13. [Project Directory Structure](#)
 14. [Known Issues & Fixes Applied](#)
 15. [Ports & Network Reference](#)
-

1. Project Overview

FundScan is an enterprise-grade **autonomous digital worker** built by Stradit AI. It automatically ingests, processes, and analyzes data for mutual funds and ETFs, then delivers actionable, board-ready insights combining **quantitative performance metrics** with **investor sentiment signals**.

What it does

- Continuously tracks NAV prices, benchmark returns, fund flows, and fund holdings
- Calculates 21+ performance and risk metrics (Sharpe, Sortino, Alpha, Beta, Tracking Error, Style Drift, etc.)
- Analyzes public investor sentiment from news, SEC filings, and social media
- Detects divergence between fund performance and investor sentiment
- Raises automated risk flags for governance review
- Generates AI-powered board-ready PDF and Excel reports
- Exposes all functionality via a 34-endpoint REST API
- Presents an interactive Next.js dashboard for analysts

Who uses it

Role	Usage
Fund Analysts	Monitor fund performance and sentiment in real-time
Risk Officers	Review active risk flags and governance metrics
Board Members	Receive monthly board-ready PDF/Excel reports
Data Engineers	Manage pipeline runs and data quality
Developers	Consume the REST API to build downstream products

2. Business Problem & Objectives

The Problem

Traditional fund analysis is fragmented:

- Performance data and investor sentiment are analyzed in separate silos
- Risk flags are raised manually after problems escalate
- Board reports are produced manually — slow, error-prone, and inconsistent
- No single system provides real-time correlation between performance and public perception

Why FundScan

FundScan solves this by being an **autonomous, end-to-end pipeline** that:

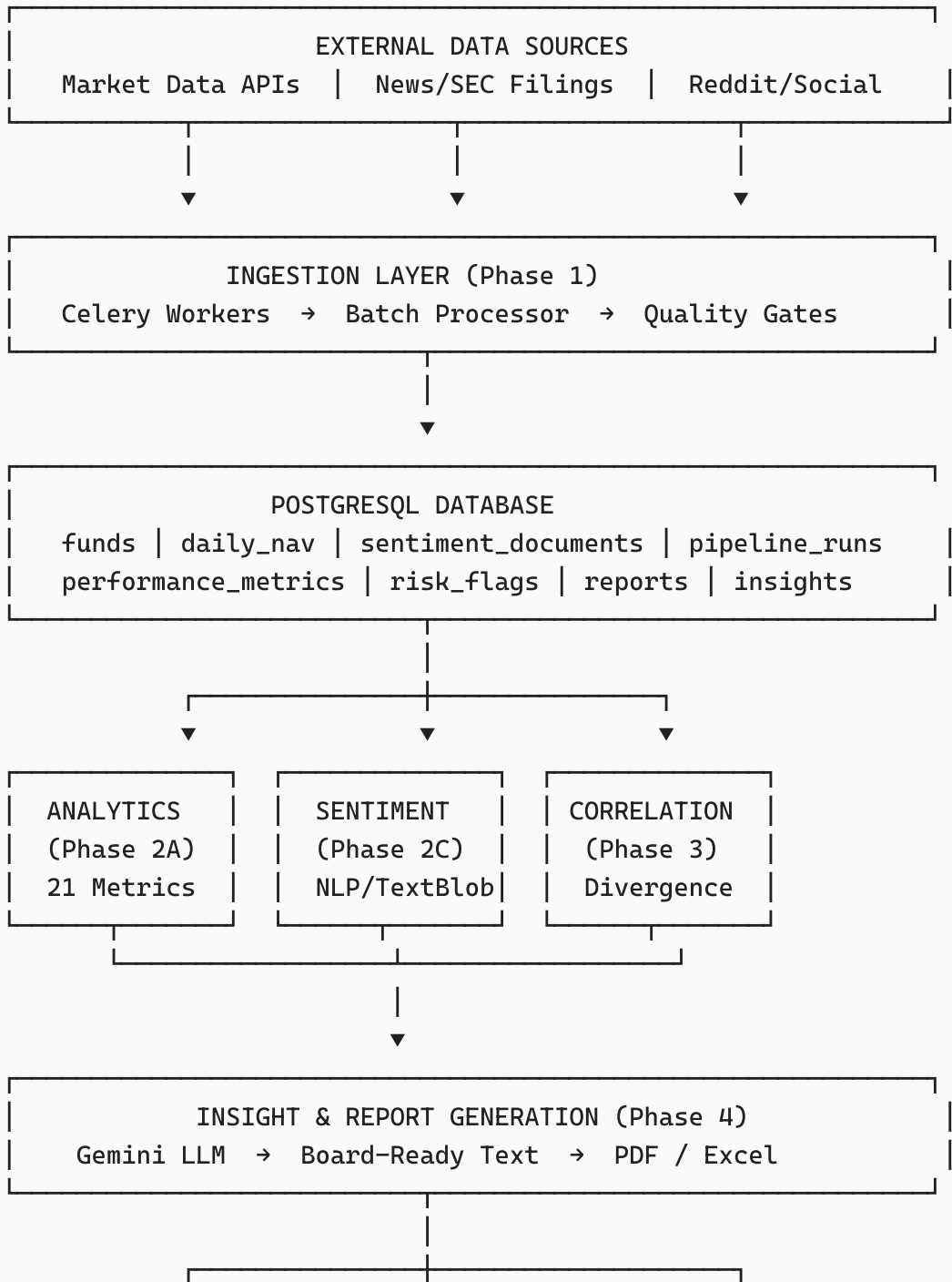
- Eliminates manual data collection and report writing
- Surfaces early-warning signals through Performance–Sentiment divergence detection
- Standardizes governance metrics across all funds in a single database
- Delivers AI-generated, board-compliant analysis on a fixed schedule

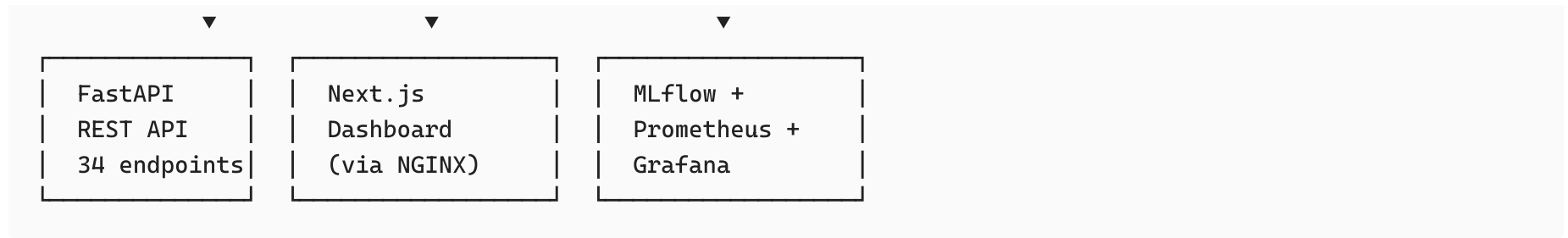
Objectives

1. Automate ingestion of quantitative and qualitative fund data daily
 2. Compute 21 standardized metrics per fund per period (1M, 3M, 6M, 1Y, 3Y, YTD)
 3. Detect sentiment-performance divergences above configurable thresholds
 4. Generate board-ready reports (PDF + Excel) automatically every month
 5. Expose all data via a versioned REST API (/api/v1)
 6. Track all pipeline runs, model metrics, and experiments via MLflow
-

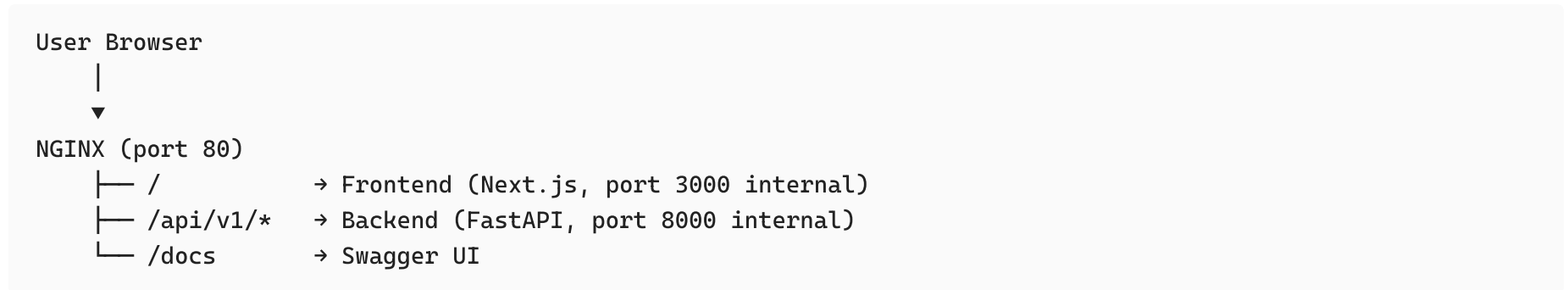
3. System Architecture

High-Level Architecture

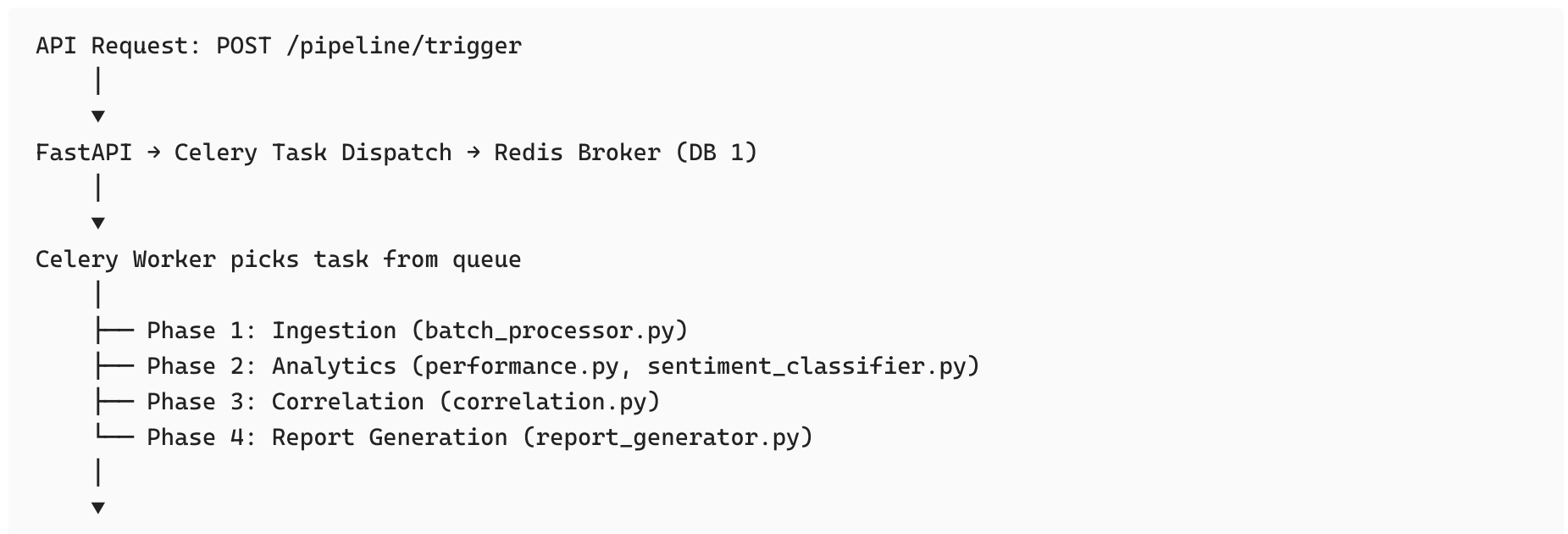




Request Flow



Async Task Flow



Results stored → PostgreSQL
MLflow run logged → PostgreSQL (mlflow schema)

4. Technology Stack

Backend

Component	Technology	Version	Why
API Framework	FastAPI	0.115.0	Async-native, auto-generates OpenAPI docs
ASGI Server	Uvicorn	0.30.6	High-performance async server
ORM	SQLAlchemy (asyncio)	2.0.35	Async DB support, powerful query builder
DB Migrations	Alembic	1.13.2	Schema versioning
DB Driver	asyncpg	0.30.0	Async PostgreSQL driver
DB Driver (sync)	psycopg2-binary	≥2.9.9	Celery Beat compatibility
Async Queue	Celery	5.4.0	Distributed task queue for pipeline
Broker	Redis	5.1.0	Message broker for Celery
Data Processing	Pandas	2.2.3	Fund data manipulation
Numerics	NumPy	2.1.1	Metric calculations
Statistical	SciPy	1.14.1	Regression, correlation
NLP (basic)	TextBlob	0.18.0	Baseline sentiment scoring
NLP (advanced)	spaCy	3.8.11	Named entity recognition
NLP (corpus)	NLTK	3.9.1	Tokenization, stop words
LLM Integration	google-genai	1.5.0	Gemini Flash for insight generation
PDF Generation	pdfkit + xhtml2pdf	1.0.0 / 0.2.16	Board-ready PDF reports

Component	Technology	Version	Why
Excel Generation	openpyxl	3.1.5	Board-ready Excel reports
Templates	Jinja2	3.1.4	HTML → PDF template engine
Experiment Tracking	MLflow	2.19.0	Pipeline run tracking & model registry
Observability	prometheus-client	0.21.0	HTTP metrics for Grafana
Validation	Pydantic	2.9.0	Request/response validation
Config	pydantic-settings	2.5.0	Env var configuration
Testing	pytest + pytest-asyncio	8.3.3 / 0.24.0	Async test suite
HTTP Client	httpx	0.28.1	Async HTTP for LLM calls
Logging	Loguru	0.7.2	Structured logging
Env Vars	python-dotenv	1.0.1	.env file loading
Runtime	Python	3.11 (Docker) / 3.13 (local)	

Frontend

Component	Technology	Version	Why
Framework	Next.js	^14.2.18	SSR + file-based routing
Language	TypeScript	^5.6.2	Type safety
UI Runtime	React	^18.3.1	Component model
Charts	Recharts	^2.12.7	Fund performance charts
State Management	Zustand	^4.5.5	Lightweight global state
HTTP	Axios	^1.7.7	API calls to backend
Icons	Lucide React + react-icons	latest	UI icons
Notifications	react-hot-toast	^2.4.1	User feedback

Component	Technology	Version	Why
Date Handling	date-fns	^3.6.0	Period formatting
Styling	Tailwind CSS	^3.4.13	Utility-first CSS
Testing	Vitest	^2.0.5	Unit/component tests

Infrastructure

Component	Technology	Version
Reverse Proxy	NGINX	1.27-alpine
Database	PostgreSQL	16-alpine
Cache / Broker	Redis	7-alpine
Experiment Tracking	MLflow	Custom image
Metrics Scraper	Prometheus	v2.52.0
System Metrics	Node Exporter	v1.8.2
DB Metrics	Postgres Exporter	v0.15.0
Cache Metrics	Redis Exporter	v1.55.0
Dashboards	Grafana	11.3.0
Container Orchestration	Docker Compose	v2

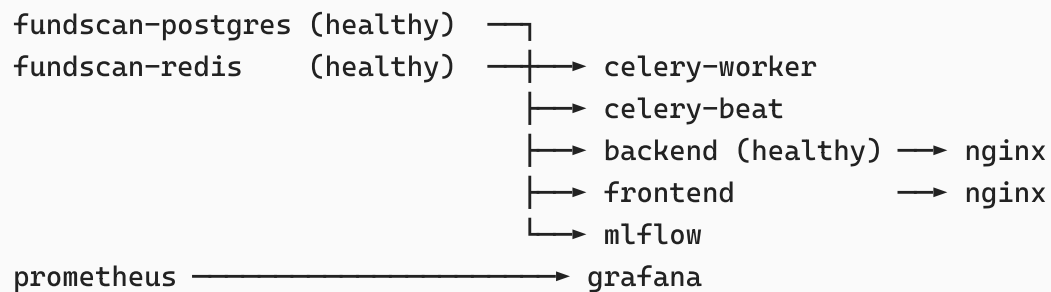
5. Infrastructure & Docker Services

The entire application runs as **13 Docker containers** orchestrated by Docker Compose, all on an isolated bridge network called `fundscan-network`.

Container Inventory

#	Container	Image	Role	Internal Port
1	fundscan-nginx	nginx:1.27-alpine	Reverse proxy & gateway	80
2	fundscan-backend	Python 3.11 / FastAPI	REST API server	8000
3	fundscan-frontend	Node 20 / Next.js	Dashboard UI	3000
4	fundscan-celery-worker	Same as backend	Async pipeline executor	—
5	fundscan-celery-beat	Same as backend	Recurring job scheduler	—
6	fundscan-postgres	postgres:16-alpine	Primary relational database	5432
7	fundscan-mlflow	fundscan-mlflow	Experiment tracking server	5007
8	fundscan-redis	redis:7-alpine	Cache & message broker	6379
9	fundscan-prometheus	prom/prometheus:v2.52.0	Metrics collector	9090
10	fundscan-node-exporter	prom/node-exporter:v1.8.2	System metrics (CPU/RAM/disk)	9100
11	fundscan-postgres-exporter	postgres-exporter:v0.15.0	DB metrics	9187
12	fundscan-redis-exporter	redis_exporter:v1.55.0	Redis metrics	9121
13	fundscan-grafana	grafana:11.3.0	Dashboard & alerting	3000

Container Dependencies (startup order)



Health Checks

- **PostgreSQL:** `pg_isready -U fundscan -d fundscan_db` every 10s
- **Redis:** `redis-cli ping` every 10s
- **Backend:** `curl -f http://localhost:8000/api/v1/health` every 30s
- **Grafana:** `wget http://localhost:3000/api/health` every 15s

Volumes

Volume	Purpose
<code>fundscan-pgdata</code>	PostgreSQL data persistence
<code>fundscan-redis-data</code>	Redis persistence
<code>fundscan-reports</code>	Generated PDF/Excel reports
<code>fundscan-mlflow-artifacts</code>	MLflow model artifacts
<code>fundscan-prometheus-data</code>	Prometheus time-series data
<code>fundscan-grafana-data</code>	Grafana dashboards & config

6. Database Design

Database: PostgreSQL 16
User: `fundscan`
Database Name: `fundscan_db`
Init Script: `scripts/db/001_init_schema.sql`

Schema Overview

MASTER DATA

- └─ benchmarks - Market benchmarks (S&P 500, etc.)
- └─ peer_groups - Fund category peer groups
- └─ funds - Core fund master data

PHASE 1 - RAW DATA

- └─ daily_nav - Daily NAV prices & returns
- └─ benchmark_daily_returns - Daily benchmark returns
- └─ fund_flows - Monthly inflow/outflow data
- └─ fund_expenses - Annual expense ratios & fees
- └─ fund_holdings - Quarterly sector weights & top holdings
- └─ sentiment_documents - Raw text from news/SEC/social media

PHASE 2 - COMPUTED METRICS

- └─ performance_metrics - 21 metrics per fund per period
- └─ governance_metrics - Expense drag, style drift, AUM trend
- └─ sentiment_signals - Aggregated sentiment scores per fund

PHASE 3 - ANALYSIS

- └─ correlation_results - Performance-Sentiment correlation
- └─ divergence_results - Divergence detection per fund
- └─ risk_flags - Active governance risk alerts

PHASE 4 - OUTPUT

- └─ insights - AI-generated text summaries
- └─ reports - Board-ready report metadata

PIPELINE

- └─ pipeline_runs - Run tracking & audit trail
- └─ data_quality_issues - DQ check issue tracker

Key Tables

funds

```

fund_id          VARCHAR(20) PRIMARY KEY
fund_name        VARCHAR(300) NOT NULL
ticker           VARCHAR(20)
category         VARCHAR(100)
fund_type        VARCHAR(20)  -- 'Mutual Fund' or 'ETF'
inception_date   DATE
benchmark_id     → benchmarks
peer_group_id    → peer_groups
manager_name     VARCHAR(200)
strategy_description TEXT

```

daily_nav

```

id              SERIAL PRIMARY KEY
fund_id         → funds
date            DATE
nav             NUMERIC(12,4)
daily_return    NUMERIC(10,6)
total_return    NUMERIC(10,6)
volume          BIGINT
UNIQUE(fund_id, date)

```

performance_metrics

```

fund_id         → funds
period          VARCHAR(10)  -- '1M', '3M', '6M', '1Y', '3Y', 'YTD'
as_of_date      DATE
total_return     NUMERIC(10,4)
relative_return  NUMERIC(10,4)
excess_return    NUMERIC(10,4)
peer_percentile  INTEGER
volatility       NUMERIC(10,4)
max_drawdown     NUMERIC(10,4)

```

```
sharpe_ratio      NUMERIC(8,4)
sortino_ratio     NUMERIC(8,4)
tracking_error    NUMERIC(10,4)
batting_average   NUMERIC(6,4)
performance_persistence VARCHAR(20)
UNIQUE(fund_id, period, as_of_date)
```

sentiment_documents

```
doc_id           VARCHAR(30) PRIMARY KEY
fund_id          → funds
date            DATE
source           VARCHAR(200)
source_type      VARCHAR(50)  -- 'news', 'sec_filing', 'reddit'
title            VARCHAR(500)
body_text        TEXT
sentiment_score  NUMERIC(5,3) -- -1.0 to +1.0
sentiment_label  VARCHAR(20)  -- 'POSITIVE', 'NEGATIVE', 'NEUTRAL'
topics           JSONB
risk_flags       JSONB
```

risk_flags

```
flag_id          VARCHAR(30) PRIMARY KEY  -- 'FLG-000001'
fund_id          → funds
flag_type        VARCHAR(50)
severity         VARCHAR(20)  -- 'LOW', 'MEDIUM', 'HIGH', 'CRITICAL'
title            VARCHAR(300)
description       TEXT
status           VARCHAR(20)  -- 'ACTIVE', 'ACKNOWLEDGED', 'RESOLVED'
acknowledged_by  VARCHAR(100)
resolution_notes TEXT
```

Important Fix: The `risk_flag_seq` sequence must be created BEFORE the `risk_flags` table. This was a bug in the original schema that caused PostgreSQL initialization to fail; it has been fixed in `001_init_schema.sql`.

7. The 6-Phase Autonomous Pipeline

The pipeline runs end-to-end either on a schedule (via Celery Beat) or on-demand (via `POST /api/v1/pipeline/trigger`).

Phase 1 – Data Ingestion

What: Pulls raw quantitative and sentiment data into PostgreSQL.

Where: `backend/app/services/ingestion/`

Key files:

- `performance.py` – Fetches daily NAV and benchmark data
- `sentiment.py` – Fetches news articles, SEC filings, social media posts
- `batch_processor.py` – Parallelizes ingestion across multiple funds
- `batch_tasks.py` – Celery tasks for batch execution
- `quality_gates.py` – Validates data completeness and freshness

Schedule: Daily at 2:00 AM UTC

```
Trigger → batch_processor.py (groups funds into batches of N)
        → process_fund_batch (Celery task per batch)
        → quality_gates.py (reject bad data)
        → INSERT INTO daily_nav, sentiment_documents
```

Phase 2A – Performance Metrics Engine

What: Computes 21 standardized performance and risk metrics for every fund and every time period.

Where: `backend/app/services/analytics/performance.py` + `performance_math.py`

Metrics Calculated:

Metric	Description
Total Return	Cumulative return over period
Relative Return	Return vs benchmark
Excess Return	Return vs peer group median
Peer Percentile	Ranking within peer group
Annualized Volatility	Standard deviation of daily returns
Max Drawdown	Largest peak-to-trough decline
Sharpe Ratio	Risk-adjusted return (rf = 0)
Sortino Ratio	Downside risk-adjusted return
Tracking Error	Std dev of excess returns vs benchmark
Batting Average	% of months outperforming benchmark
Alpha	Jensen's Alpha (CAPM regression)
Beta	Market sensitivity (CAPM regression)
Performance Persistence	Consistency label (Consistent/Variable/Poor)

Time Periods: 1M, 3M, 6M, 1Y, 3Y, YTD

Schedule: Weekly (Mondays at 4:00 AM UTC)

Phase 2B – Governance Metrics

Where: backend/app/services/analytics/governance.py

Computes:

- Expense ratio drag (basis points vs peer group)

- Style drift detection (sector weight shift analysis)
- AUM trend (net flows in/out)
- Flow-adjusted returns
- Benchmark appropriateness score
- Alert frequency count

Phase 2C – Sentiment Analysis Engine

Where: `backend/app/services/analytics/sentiment_classifier.py` (25KB — largest service file)

Pipeline within pipeline:

1. Text preprocessing (NLTK tokenization, stopword removal)
2. Named Entity Recognition via spaCy (identify fund names, companies, locations)
3. Baseline scoring via TextBlob polarity
4. Domain-specific keyword weighting for financial vocabulary
5. Topic classification (JSONB stored)
6. Reputational risk scoring
7. Aggregation into `sentiment_signals` table

Phase 3 – Correlation & Divergence Detection

Where: `backend/app/services/analytics/correlation.py` + `correlation_math.py`

What it does:

1. Aligns performance metric time-series with sentiment score time-series
2. Computes Pearson correlation coefficient across configurable time horizons
3. Flags **divergence** when performance is high but sentiment is negative (or vice versa)
4. Classifies divergence type and severity
5. Raises `risk_flags` records automatically for high-severity divergences

Phase 4A – AI Insight Generation

Where: `backend/app/services/analytics/insight_generator.py`

Uses **Google Gemini Flash** (`gemini-2.0-flash`) to generate:

- Executive summary of fund performance
- Sentiment assessment
- Risk assessment
- Key takeaways (JSON array)
- Board-ready compliant narrative

Requires `LLM_API_KEY` in `.env` .

Phase 4B – Report Generation

Where: `backend/app/services/reporting/report_generator.py`

Generates:

- **PDF** reports (via `pdfkit` + `wkhtmltopdf` + Jinja2 HTML templates)
- **Excel** reports (via `openpyxl`, multi-sheet layout)
- Stored to `/app/generated_reports/` volume
- Paths saved in `reports` table

Schedule: Monthly (1st of each month at 6:00 AM UTC)

Phase 5 – Dashboard & API

What: Exposes all computed data via REST API; serves interactive Next.js dashboard.

Where: `backend/app/api/v1/endpoints/` + `frontend/src/`

Phase 6 – Validation & Monitoring

What: End-to-end pipeline health, observability, and data quality tracking.

Where: MLflow + Prometheus + Grafana

8. Backend – API Reference

Base URL: `http://localhost/api/v1` (via NGINX)

Direct URL: `http://localhost:8000/api/v1`

Swagger UI: `http://localhost/docs`

OpenAPI JSON: `http://localhost/openapi.json`

API Modules & Endpoints

Funds (`funds.py`)

Method	Endpoint	Description
GET	<code>/funds</code>	List all funds with filters
GET	<code>/funds/{fund_id}</code>	Get fund detail
POST	<code>/funds</code>	Create a new fund record
PUT	<code>/funds/{fund_id}</code>	Update fund record
DELETE	<code>/funds/{fund_id}</code>	Delete fund record

Metrics (`metrics.py`)

Method	Endpoint	Description
GET	<code>/funds/{fund_id}/metrics/performance</code>	Performance metrics (by period)
GET	<code>/funds/{fund_id}/metrics/governance</code>	Governance metrics

Method	Endpoint	Description
GET	/funds/{fund_id}/nav	Daily NAV time-series
POST	/funds/{fund_id}/nav	Ingest NAV data points

Sentiment (sentiment.py)

Method	Endpoint	Description
GET	/funds/{fund_id}/sentiment/trends	Sentiment trend time-series
GET	/funds/{fund_id}/sentiment/documents	Raw sentiment documents
POST	/funds/{fund_id}/sentiment/documents	Ingest a sentiment document

Correlation (correlation.py)

Method	Endpoint	Description
GET	/funds/{fund_id}/correlation	Correlation results
GET	/funds/{fund_id}/divergence	Divergence detection results

Risk Flags (risk_flags.py)

Method	Endpoint	Description
GET	/risk-flags	All active risk flags
GET	/risk-flags/{flag_id}	Single flag detail
POST	/risk-flags	Create a risk flag manually
PATCH	/risk-flags/{flag_id}	Acknowledge or resolve a flag

Insights (insights.py)

Method	Endpoint	Description
GET	/funds/{fund_id}/insights	Get AI-generated insights
POST	/funds/{fund_id}/insights/generate	Trigger insight generation (LLM)

Reports (reporting.py)

Method	Endpoint	Description
GET	/reports	List all reports
GET	/reports/{report_id}	Get report metadata
POST	/reports/generate	Trigger report generation
GET	/reports/{report_id}/export/pdf	Download PDF
GET	/reports/{report_id}/export/excel	Download Excel

Pipeline (pipeline.py)

Method	Endpoint	Description
GET	/pipeline/status	Current pipeline status
POST	/pipeline/trigger	Trigger a full pipeline run
GET	/pipeline/runs	List all pipeline runs
GET	/pipeline/runs/{run_id}	Run detail + stage breakdown
GET	/data-quality/status	Data quality check summary
GET	/data-quality/issues	Active DQ issues

Autonomous (`autonomous.py`)

Method	Endpoint	Description
POST	<code>/autonomous/run</code>	Trigger autonomous pipeline

TTS (`tts.py`)

Method	Endpoint	Description
POST	<code>/tts/synthesize</code>	Text-to-speech for report narration

Health (`health.py`)

Method	Endpoint	Description
GET	<code>/health</code>	API health check (used by Docker)

Prometheus Metrics

Path	Description
<code>/metrics</code>	Prometheus-format HTTP request counters & latency histograms

Request/Response Format

- All requests and responses use `application/json`
- Standard error format: `{"detail": "Error message"}`
- Timestamps: ISO 8601 UTC

Middleware

- **CORS:** Allows all origins (*) — tighten in production
- **Prometheus Middleware:** Tracks fundscan_http_requests_total and fundscan_http_request_latency_seconds counters per method/endpoint

9. Frontend – Dashboard & UI

Framework: Next.js 14 (App Router) + TypeScript + Tailwind CSS

URL: http://localhost/ (via NGINX) or http://localhost:3002 (direct)

Directory Structure

```
frontend/src/
├── app/                - Next.js App Router pages
├── components/
│   ├── common/        - Shared UI components
│   ├── dashboard/     - Fund performance dashboard components
│   ├── kyc/           - KYC-related views
│   └── reports/       - Report viewer components
├── services/          - Axios API client wrappers
├── store/              - Zustand global state stores
├── lib/                - Utility helpers
└── utils/              - Date, number formatting
```

Key UI Components

Component Area	What it shows
Dashboard	Fund list, performance metrics, charts (via Recharts), sentiment trends

Component Area	What it shows
KYC	Fund-level KYC / investor sentiment profile
Reports	Board-ready report viewer, PDF/Excel download buttons
Risk Flags	Active governance alerts with severity indicators

API Integration

- All API calls go through `services/` using Axios
 - `NEXT_PUBLIC_API_URL` env var sets the API base URL (defaults to `http://localhost`)
 - State is managed globally via Zustand stores in `store/`
-

10. Observability Stack

MLflow – Experiment Tracking & Model Registry

URL: `http://localhost:5007`

Where: `backend/app/services/mlflow_logging.py` + `backend/app/core/mlflow_config.py`

Backend Store: PostgreSQL (same `fundscan_db`, mlflow schema tables)

Artifact Store: Docker volume `fundscan-mlflow-artifacts`

What is logged:

- Every pipeline run → MLflow run with params (stages, status, duration)
- Every analytics computation → MLflow run with 21 performance metrics
- Model metadata registered in MLflow Model Registry

Key functions:

- `log_pipeline_run()` – logs pipeline execution as an MLflow run

- `log_analytics_metrics_run()` – logs metric computation + registers model
- `get_latest_model_version()` – retrieves latest production model version

Prometheus – Metrics Collection

URL: `http://localhost:9095`

Config: `monitoring/prometheus/prometheus.yml`

Scraped targets:

- `fundscan-backend:8000/metrics` – FastAPI request metrics
- `fundscan-node-exporter:9100` – System metrics (CPU, RAM, disk)
- `fundscan-postgres-exporter:9187` – PostgreSQL metrics
- `fundscan-redis-exporter:9121` – Redis metrics

Custom Application Metrics:

- `fundscan_http_requests_total` (Counter) – labeled by method + endpoint
- `fundscan_http_request_latency_seconds` (Histogram) – labeled by method + endpoint

Grafana – Dashboards & Alerting

URL: `http://localhost:3005`

Credentials: `admin / admin` (change in production)

Config: `monitoring/grafana/provisioning/`

Pre-provisioned dashboards in `monitoring/grafana/dashboards/` .

11. Environment Configuration

Copy `.env.example` to `.env` before running:

Copy-Item .env.example .env

Full .env Reference

```
# — General —————
ENVIRONMENT=development          # or 'production'
LOG_LEVEL=info                   # debug|info|warning|error

# — PostgreSQL —————
POSTGRES_USER=fundscan
POSTGRES_PASSWORD=fundscan_secret # ⚠ Change in production
POSTGRES_DB=fundscan_db
DATABASE_URL=postgresql+asyncpg://fundscan:fundscan_secret@fundscan-postgres:5432/fundscan_db

# — Redis —————
REDIS_URL=redis://fundscan-redis:6379/0
CELERY_BROKER_URL=redis://fundscan-redis:6379/1
CELERY_RESULT_BACKEND=redis://fundscan-redis:6379/2

# — LLM (Required for Phase 4A Insights) —————
LLM_API_KEY=your-gemini-api-key-here # ⚠ Required for AI insights
LLM_MODEL_NAME=gemini-2.0-flash
LLM_TEMPERATURE=0.3
LLM_MAX_TOKENS=4096

# — Backend —————
BACKEND_HOST=0.0.0.0
BACKEND_PORT=8000
SECRET_KEY=change-this-to-a-secure-random-string # ⚠ Change in production
ACCESS_TOKEN_EXPIRE_MINUTES=60

# — Frontend —————
REACT_APP_API_BASE_URL=http://localhost:8080/api/v1
```

```
# — Pipeline Scheduling —————  
SCHEDULE_DAILY_INGESTION=0 2 * * *      # 2 AM UTC daily  
SCHEDULE_WEEKLY_METRICS=0 4 * * 1      # 4 AM UTC Monday  
SCHEDULE_MONTHLY_REPORTS=0 6 1 * *      # 6 AM UTC 1st of month  
  
# — Report Generation —————  
REPORT_OUTPUT_DIR=/app/generated_reports  
PDF_PAGE_SIZE=A4  
EXCEL_TEMPLATE_DIR=/app/templates/excel
```

Security note for production:

- Change `POSTGRES_PASSWORD`, `SECRET_KEY`
- Set a real `LLM_API_KEY`
- Restrict CORS `allow_origins` in `main.py`
- Change Grafana admin password

12. Developer Setup Guide

Prerequisites

- [Docker Desktop](#) (v20+, Docker Compose v2)
- [Git](#)
- [Python 3.11+](#) (for local backend dev)
- [Node.js 20+](#) (for local frontend dev)

Option A: Full Stack via Docker (Recommended)

```
# 1. Clone
git clone https://github.com/Stradit/fundscan.git
cd fundscan

# 2. Create .env
Copy-Item .env.example .env
# Open .env and set LLM_API_KEY

# 3. Start everything
docker compose up -d --build

# 4. Verify
curl http://localhost/api/v1/health
docker compose ps
```

Option B: Local Backend Development

```
# Navigate to backend
cd backend

# Create virtual environment
python -m venv .venv

# Activate (PowerShell)
.\.venv\Scripts\Activate.ps1
# If blocked: Set-ExecutionPolicy -ExecutionPolicy RemoteSigned -Scope Process

# Install dependencies
pip install -r requirements.txt
```

```
# Run FastAPI with hot-reload
uvicorn app.main:app --reload --host 0.0.0.0 --port 8000
```

Note on psycopg2-binary: On Python 3.13 with no Microsoft C++ Build Tools, the pinned version 2.9.9 fails to build. Use `psycopg2-binary>=2.9.9` (version 2.9.12 has pre-built wheels for Python 3.13 on Windows).

Option C: Local Frontend Development

```
cd frontend
npm install
npm run dev
# Accessible at http://localhost:3000
```

Running Tests

```
# Backend
cd backend
pytest

# Frontend
cd frontend
npm run test
```

Resetting Everything (Clean Slate)

```
# Stop and remove all containers + volumes (⚠ deletes all data)
```

```
docker compose down -v
```

```
# Rebuild from scratch
```

```
docker compose up -d --build
```

13. Project Directory Structure

```
fundscan/
|
├── .env.example          - Environment variable template
├── .env                  - Active environment config (git-ignored)
├── .gitignore
├── docker-compose.yml    - 13-container orchestration
├── README.md
|
├── backend/             - FastAPI application
|   ├── Dockerfile        - Python 3.11-bookworm image
|   ├── requirements.txt   - Python dependencies
|   └── app/
|       ├── main.py        - App factory, routers, middleware
|       ├── api/v1/endpoints/
|           ├── funds.py    - Fund CRUD
|           ├── metrics.py  - Performance & governance metrics
|           ├── sentiment.py - Sentiment documents & trends
|           ├── correlation.py - Correlation & divergence
|           ├── risk_flags.py - Risk flag management
|           ├── insights.py - AI insight generation
|           ├── reporting.py - Report export endpoints
|           ├── pipeline.py - Pipeline trigger & monitoring
|           └── autonomous.py - Autonomous run endpoint
```

- └─ tts.py - Text-to-speech endpoint
- └─ health.py - Health check
- └─ core/
 - └─ celery_app.py - Celery config + beat schedule
 - └─ database.py - SQLAlchemy async engine
 - └─ mlflow_config.py - MLflow URI & experiment config
 - └─ schema_bootstrap.py - Ensures DB schema on startup
- └─ models/ - SQLAlchemy ORM models
 - └─ __init__.py - Exports all models
 - └─ fund.py - Fund, Benchmark, PeerGroup
 - └─ metrics.py - DailyNav, PerformanceMetrics, etc.
 - └─ sentiment.py - SentimentDocument, SentimentSignal
 - └─ correlation.py - CorrelationResult, DivergenceResult
 - └─ reporting.py - Insight, Report
 - └─ pipeline.py - PipelineRun, DataQualityIssue
- └─ schemas/ - Pydantic request/response schemas
- └─ services/
 - └─ mlflow_logging.py - MLflow run & metric logging
 - └─ ingestion/ - Phase 1: Data ingestion
 - └─ performance.py - NAV & benchmark fetch
 - └─ sentiment.py - Text document fetch
 - └─ batch_processor.py - Parallel batch ingestion
 - └─ batch_tasks.py - Celery batch tasks
 - └─ quality_gates.py - Data validation
 - └─ run.py - Ingestion orchestrator
 - └─ tasks.py - Celery task definitions
 - └─ analytics/ - Phases 2-4: Core analytics
 - └─ performance.py - Performance metrics orchestrator
 - └─ performance_math.py - Sharpe, Sortino, etc.
 - └─ governance.py - Governance metrics
 - └─ governance_math.py - Style drift, expense drag
 - └─ sentiment.py - Sentiment signal aggregator
 - └─ sentiment_classifier.py - NLP pipeline (largest file)
 - └─ sentiment_math.py - Sentiment scoring math
 - └─ sentiment_evaluation.py - Scoring evaluation

- └─ nlp_processor.py - spaCy NER processor
 - └─ correlation.py - Correlation engine
 - └─ correlation_math.py - Pearson & divergence math
 - └─ insight_generator.py - Gemini LLM integration
 - └─ reporting/ - Phase 4B: Reports
 - └─ report_generator.py - PDF/Excel generator
 - └─ tasks.py - Monthly report Celery task
 - └─ tasks.py - Weekly metrics Celery task
 - └─ templates/ - Jinja2 HTML templates for PDF
 - └─ frontend/ - Next.js dashboard
 - └─ Dockerfile - Node 20-alpine image
 - └─ package.json
 - └─ next.config.js
 - └─ tailwind.config.js
 - └─ src/
 - └─ app/ - Next.js App Router
 - └─ components/
 - └─ common/ - Shared UI components
 - └─ dashboard/ - Performance dashboard
 - └─ kyc/ - KYC views
 - └─ reports/ - Report viewer
 - └─ services/ - Axios API clients
 - └─ store/ - Zustand state
 - └─ utils/ - Helpers & formatters
 - └─ scripts/
 - └─ db/
 - └─ 001_init_schema.sql - Database schema (auto-run on first start)
 - └─ deployment/ - Deployment scripts
 - └─ nginx/
 - └─ nginx.conf - NGINX reverse proxy config
 - └─ monitoring/

```
| | | | prometheus/
| | | | | | | | prometheus.yml      - Scrape targets config
| | | | | | | | alerts.yml         - Alert rules
| | | | | grafana/
| | | | | | | | provisioning/      - Auto-provisioned datasources
| | | | | | | | dashboards/       - Pre-built dashboard JSON
| | | |
| | | | | docker/
| | | | | | | | mlflow/
| | | | | | | | | | Dockerfile      - Custom MLflow image
| | | |
| | | | | data_pipeline/          - Standalone ETL workflows
| | | | | | | | ingestion/
| | | | | | | | processing/
| | | | | | | | orchestration/
```

14. Known Issues & Fixes Applied

Issue 1: PostgreSQL container unhealthy on first start

Root Cause: `scripts/db/001_init_schema.sql` referenced sequence `risk_flag_seq` in the `risk_flags` table's `DEFAULT` clause before creating the sequence.

Fix Applied: Moved `CREATE SEQUENCE IF NOT EXISTS risk_flag_seq START 1;` to appear **before** the `CREATE TABLE risk_flags` statement.

Before (broken):

```
CREATE TABLE IF NOT EXISTS risk_flags (
  flag_id VARCHAR(30) PRIMARY KEY DEFAULT 'FLG-' || LPAD(nextval('risk_flag_seq')::TEXT, 6, '0'),
  ...
```

```
);  
CREATE SEQUENCE IF NOT EXISTS risk_flag_seq START 1; -- TOO LATE!
```

After (fixed):

```
CREATE SEQUENCE IF NOT EXISTS risk_flag_seq START 1; -- Must come first  
CREATE TABLE IF NOT EXISTS risk_flags (  
    flag_id VARCHAR(30) PRIMARY KEY DEFAULT 'FLG-' || LPAD(nextval('risk_flag_seq')::TEXT, 6, '0'),  
    ...  
);
```

Issue 2: pip install fails for psycpg2-binary==2.9.9 on Python 3.13 / Windows

Root Cause: No pre-compiled wheel for psycpg2-binary 2.9.9 exists for Python 3.13 on Windows. Building from source requires Microsoft C++ Build Tools.

Fix Applied: Changed requirements.txt from psycpg2-binary==2.9.9 to psycpg2-binary>=2.9.9, allowing pip to install version 2.9.12 which has a pre-built Windows wheel for Python 3.13.

Issue 3: .env file missing

Root Cause: .env is git-ignored and must be created manually from .env.example.

Fix: Run Copy-Item .env.example .env before first docker compose up.

15. Ports & Network Reference

All FundScan ports are chosen to avoid conflicts with other Stradit projects (PortfolioQ, etc.).

Host Port	Service	URL
80	NGINX Gateway	http://localhost
80 → /api/v1	Backend API (via NGINX)	http://localhost/api/v1
80 → /docs	Swagger UI (via NGINX)	http://localhost/docs
8000	Backend (direct)	http://localhost:8000
3002	Frontend (direct)	http://localhost:3002
5434	PostgreSQL (direct)	localhost:5434
6381	Redis (direct)	localhost:6381
5007	MLflow UI	http://localhost:5007
9095	Prometheus	http://localhost:9095
3005	Grafana	http://localhost:3005
9100	Node Exporter	http://localhost:9100
9187	Postgres Exporter	http://localhost:9187
9121	Redis Exporter	http://localhost:9121

Ports reserved by other Stradit services (do NOT use):

- 8001 (MCP Sandbox), 3000/3001 (Grafana), 5432/5433 (PortfolioQ Postgres), 6379/6380 (PortfolioQ Redis), 9090 (Prometheus), 5001 (MLflow), 5000 (PortfolioQ API)

Document prepared by Stradit AI Development Team.

Last updated: August 2026